

Ejercicios Fundamentales de Visión por Computador Clásica: Representación Matricial, Convoluciones, Segmentación por Color y Filtrado de Ruido

Entregable práctico – Guía 1, curso Visión por Computador Avanzada

Milton Alvarado

mealvarado@elpoli.edu.co

Doctorado en Innovación e Ingeniería, Politécnico Colombiano Jaime Isaza Cadavid

16 de septiembre de 2026

Resumen

Este documento presenta la solución a los cuatro retos prácticos de visión por computador clásica planteados en la Guía 1 del curso *Visión por Computador Avanzada* (Doctorado en Innovación e Ingeniería): (1) representación matricial de una imagen sintética y caracterización estadística manual frente a vectorizada con Numpy; (2) convoluciones con kernels de bordes (Sobel) y de suavizado (gaussiano) implementadas celda a celda en una hoja de cálculo; (3) extracción y caracterización estadística por color de regiones de interés en una imagen a color; y (4) eliminación de ruido impulsivo (*salt & pepper*) mediante filtrado por convolución, comparando cuatro implementaciones manuales (media, moda, mediana, gaussiana) con sus equivalentes de OpenCV. Cada reto se resolvió con código e implementaciones propias del autor, verificado de forma independiente, y se acompaña de un análisis cuantitativo de los resultados y de las conclusiones solicitadas por la guía. Como valor agregado, el cuarto reto incorpora una comparación estadística formal (ANOVA de un factor y prueba post-hoc de Tukey) entre los filtros evaluados, un primer ensayo de la línea de trabajo doctoral del autor orientada a evaluar algoritmos SLAM mediante Diseño de Experimentos (DOE).

1. Introducción

Este documento corresponde a la entrega del componente evaluable de visión clásica del curso *Visión por Computador Avanzada* (Doctorado en Innovación e Ingeniería, Politécnico Colombiano Jaime Isaza Cadavid), de acuerdo con los cuatro retos definidos en la Guía 1 de la asignatura: (1) *Matrices a imágenes*, (2) *Convoluciones en Excel*, (3) *Colores*, y (4) *Convoluciones para filtrado*. Cada reto se resuelve siguiendo los pasos solicitados en la guía (Secciones 4.1–4.4), con implementaciones propias del autor, resultados verificados de forma independiente, y evidencia reproducible disponible en un repositorio público (Sección 3).

Adicionalmente a lo solicitado en la guía, cada reto cierra con un breve comentario sobre su relación con la línea de investigación doctoral del autor – un marco metodológico basado en Diseño de Experimentos (DOE) [Montgomery, 2019] para evaluar el desempeño de algoritmos SLAM [Cadena et al., 2016] –, dado que varios de los conceptos aquí ejercitados (medición reproducible de tiempos, caracterización estadística de variables de respuesta, comparación formal de "tratamientos" mediante ANOVA) son piezas de infraestructura reutilizables en dicho marco. Estos comentarios son deliberadamente breves: el cuerpo de este documento se centra en resolver y documentar cada

reto conforme a los criterios de evaluación de la guía, no en el marco doctoral, que queda solo esbozado como proyección en las Conclusiones (Sección 6).

2. Marco teórico

Los cuatro retos se apoyan en conceptos estándar de procesamiento digital de imágenes [Gonzalez and Woods, 2018]: la representación de una imagen como matriz de intensidades y su caracterización mediante estadísticos descriptivos (Reto 1); el operador de convolución y los kernels de Sobel y de suavizado gaussiano (Reto 2); la representación de color en el espacio BGR y su caracterización estadística por región (Reto 3); y el filtrado no lineal (mediana, moda) frente al lineal (media, gaussiana) para eliminar ruido impulsivo, evaluado aquí mediante el índice de similitud estructural (SSIM) [Wang et al., 2004] (Reto 4). Estos mismos bloques de construcción – kernels de convolución, descriptores estadísticos de región, filtrado de ruido – son además los que emplean internamente los algoritmos de SLAM visual (p. ej. ORB-SLAM2 [Mur-Artal and Tardós, 2017]) y de odometría visual [Scaramuzza and Fraundorfer, 2011], lo que motiva tratarlos aquí con detalle de primer principio en vez de como funciones de caja negra de una librería.

3. Datos y disponibilidad de código

Para los cuatro retos se trabajó de forma local (VSCode) con respaldo en control de versiones Git/GitHub, en lugar de Google Colab/Drive – sustitución explícitamente autorizada por el profesor del curso, y que cumple el mismo propósito de la guía (evidencia versionada y recuperable del proceso completo). El código completo, con instrucciones de ejecución y pruebas automatizadas, está disponible públicamente en un repositorio de GitHub [Alvarado, 2026]¹.

Los datos usados en los Retos 1, 3 y 4 (Secciones 4.1, 4.3 y 4.4) son sintéticos: la matriz aleatoria del Reto 1 se genera con semilla fija ($s = 42$) para garantizar reproducibilidad exacta, y la imagen a color usada en los Retos 3 y 4 es una escena sintética generada mediante IA generativa de imágenes (Google Gemini), diseñada para emular, con fines ilustrativos y de prueba de concepto, un banco de pruebas de calibración de cámara/color típico de un laboratorio de robótica – no corresponde a una fotografía capturada de un entorno físico real. Los resultados cuantitativos de esos retos (estadísticas por canal, análisis de filtrado) se calcularon sobre los valores de píxel reales de esa imagen, y las conclusiones sobre el comportamiento de los algoritmos evaluados son válidas independientemente del origen sintético de la imagen; la validación de dichas conclusiones sobre escenas físicas reales queda como trabajo futuro (Sección 6).

4. Desarrollo de los retos

4.1. Reto 1: Matrices a imágenes

4.1.1. Método

Siguiendo el procedimiento de la guía, se generó – sin librerías de cómputo matricial, usando únicamente la librería `random` de Python y listas de listas – una matriz de 1000×1000 valores enteros aleatorios en el rango $[0, 255]$ (equivalente a `uint8`). Sobre esa matriz se implementó, también sin librerías especializadas, una función que recorre la matriz ya aplanada a un vector unidimensional y

¹URL del repositorio pendiente de publicación final: `<URL_DEL_REPOSITORIO>`. Actualizar esta nota y la entrada `alvarado2026repo` en `referencias.bib` en cuanto el repositorio quede publicado (ver `paper/borrador_articulo.md` para el procedimiento sugerido, incluyendo el registro opcional de un DOI permanente vía Zenodo).

calcula mínimo, máximo, media y desviación estándar mediante las fórmulas explícitas $\bar{x} = \frac{1}{n} \sum_i x_i$ y $\sigma = \sqrt{\frac{1}{n} \sum_i (x_i - \bar{x})^2}$, con semilla fija $s = 42$ para garantizar reproducibilidad exacta entre corridas. Se midió con `time.time()` el tiempo total del proceso, desde la creación de la matriz hasta la obtención de estos cuatro estadísticos.

El vector aplanado se guardó en un archivo de texto plano (`.csv`) en disco. El curso autoriza explícitamente trabajar de forma local con respaldo en Git/GitHub como sustituto de subir el archivo a Google Drive y volver a leerlo desde Colab (paso solicitado por la guía); esa sustitución y el archivo generado quedan documentados en el repositorio versionado (Sección 3). Tras leer el archivo con `numpy.loadtxt` y reconstruir la matriz original, se recalcularon los mismos cuatro estadísticos con Numpy y se volvió a medir el tiempo del proceso completo, para comparar contra la versión manual. Finalmente, la matriz se visualizó como imagen en escala de grises con Matplotlib. Código completo en `retos/reto1/matrices_a_imagenes/version_final/reto_1.py` del repositorio.

4.1.2. Resultados

La Tabla 1 resume la comparación entre ambas implementaciones sobre la misma matriz (los valores estadísticos coinciden exactamente, salvo precisión numérica de punto flotante).

Cuadro 1: Comparación de estadísticos y tiempos entre la implementación manual (Python puro) y Numpy, sobre una matriz de 1000×1000 valores (semilla $s = 42$).

Métrica	Manual (Python puro)	Numpy
Mínimo	0	0
Máximo	255	255
Media	127.45	127.45
Desviación estándar	73.92	73.92
Tiempo total del proceso (s)	0.4848	0.0752

El proceso manual resultó aproximadamente $6,4\times$ más lento que el proceso con Numpy. El factor exacto de aceleración varía con el hardware (se observaron valores entre $4,3\times$ y $6,7\times$ en distintas corridas y máquinas), pero la conclusión cualitativa –Numpy sensiblemente más rápido– se sostuvo en todos los casos. La Figura 1 muestra la matriz resultante visualizada como imagen en escala de grises.

Como extensión propia más allá de lo pedido por la guía, se diseñó un experimento de un factor (tamaño de la matriz N , con $N \in \{100, 300, 500, 800, 1000\}$ y 5 repeticiones por nivel) para caracterizar el efecto de N sobre el tiempo de ejecución del proceso manual. La Figura 2 muestra la curva de respuesta resultante (media $\pm$ desviación estándar del tiempo, por nivel), consistente con el crecimiento cuadrático $O(N^2)$ esperado.

4.1.3. Análisis y conclusiones

Los estadísticos calculados manualmente y con Numpy coinciden exactamente (salvo precisión de punto flotante), lo que confirma que la implementación manual es correcta. La diferencia de tiempos ($\sim 6,4\times$ más lento en Python puro) es la esperada: Numpy vectoriza las operaciones aritméticas, evitando el recorrido explícito elemento a elemento que exige la implementación manual; el experimento adicional de la Figura 2 confirma además el crecimiento cuadrático $O(N^2)$ del proceso manual al variar el tamaño N de la matriz, mientras que Numpy escala de forma mucho más

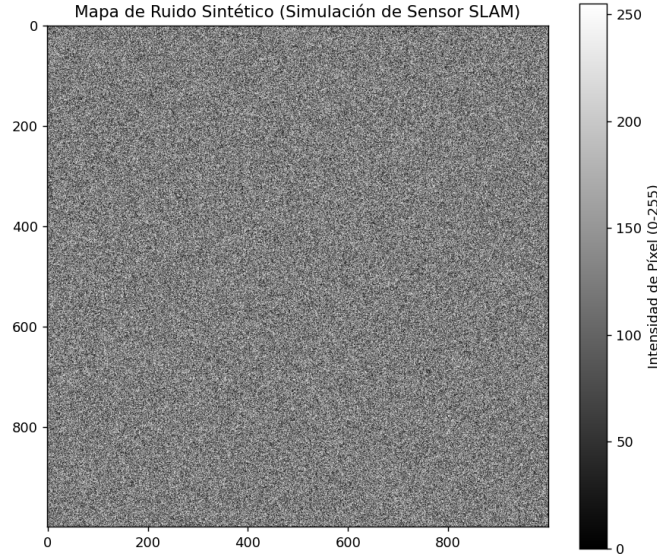


Figura 1: Matriz de 1000×1000 (semilla $s = 42$) visualizada como mapa de ruido sintético en escala de grises.

suave. Esta comparación – una variable de respuesta cuantitativa (tiempo), medida bajo condiciones controladas y reproducibles (semilla fija) – es, de forma incidental, el mismo tipo de comparación de "tratamientos" (aquí, implementaciones) que la línea de investigación doctoral del autor busca formalizar más adelante para comparar algoritmos SLAM (Sección 6).

4.2. Reto 2: Convoluciones en Excel

4.2.1. Método

Se construyó, en una hoja de cálculo, una matriz base de 30×30 celdas con una forma interna no rectangular: una letra "T", elegida porque, a diferencia de una esquina simple, combina bordes en ambas orientaciones (horizontales y verticales) con dos tipos de vértice – 4 convexos (las puntas de la barra y las esquinas inferiores del asta) y 2 cóncavos o "entrantes" (donde el asta se une a la barra) –, un tipo de unión (T) que la literatura de detección de esquinas/uniones reconoce como *feature* estructural característico de entornos contruidos (pasillos, marcos de puerta), junto a las uniones en L y en X. La matriz se generó con tres niveles aleatorios en vez de valores teóricos fijos, para introducir una variabilidad tipo ruido de sensor: fondo entre 0 y 20 (`ALEATORIO.ENTRE(0;20)`), un borde de transición de una celda entre 180 y 220 (efecto de penumbra/desenfoque), y el interior de la letra entre 220 y 255. Siguiendo la guía, las celdas se colorearon con formato condicional de escala de grises (0 = negro, 127 = gris medio, 255 = blanco), y se definieron y probaron varios kernels 3×3 : dos kernels de borde (Sobel horizontal K_x y vertical K_y) y un kernel de suavizado (gaussiano), los tres tomados de la literatura estándar de procesamiento de imágenes [Gonzalez and Woods, 2018]. Los dos Sobel se combinaron como $|G_x| + |G_y|$ para obtener el mapa de bordes. El mismo procedimiento (Sobel-H, Sobel-V, combinado) se repitió sobre una versión de la matriz suavizada previamente con el kernel gaussiano 3×3 ($\sigma \approx 0,8$), para comparar la sensibilidad del operador de Sobel al ruido de fondo con y sin un preprocesamiento de suavizado. El Algoritmo 1 describe, de forma general, la operación de filtrado por vecindad deslizante que subyace tanto a este reto como al Reto 4 (Sección 4.4): la única diferencia entre ambos es la función de agregación f

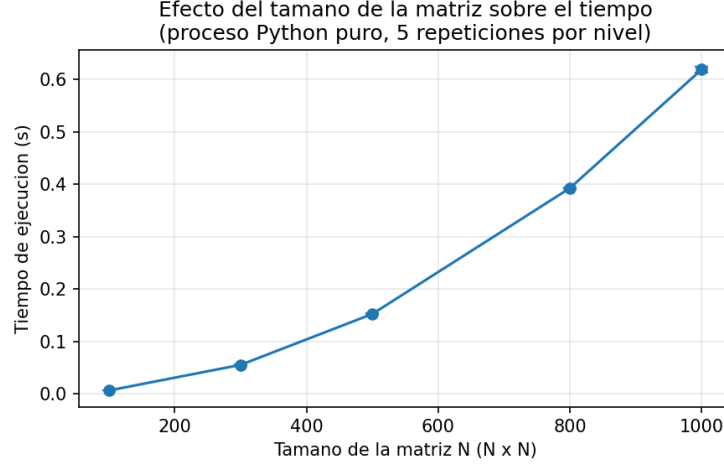


Figura 2: Efecto del tamaño de la matriz N sobre el tiempo de ejecución del proceso manual (5 repeticiones por nivel).

aplicada a cada vecindad.

Un kernel 3×3 solo puede calcularse en celdas que tengan un vecindario completo de 8 vecinas; en una matriz de 30×30 eso deja sin vecindario completo a la última fila y a la última columna si la fórmula se ancla en la esquina superior izquierda de cada vecindad (como aquí), de modo que la región con una convolución válida es de 28×28 celdas, no de 30×30 . En una primera versión de la hoja, la fórmula de **SUMPRODUCT** se extendió por error a las 30×30 celdas, incluida esa última fila/columna sin vecindario completo; se corrigió eliminando esas dos filas y dos columnas finales, que no contenían un resultado válido.

Algorithm 1 Filtrado por vecindad deslizante 3×3

Require: Imagen I de tamaño $H \times W$; función de agregación f

(f = suma ponderada por kernel K , en este reto;

$f \in \{\text{media, mediana, moda}\}$ en el Reto 4)

Ensure: Imagen filtrada I'

1: **for** $i \leftarrow 1$ **hasta** $H - 2$ **do**

2: **for** $j \leftarrow 1$ **hasta** $W - 2$ **do**

3: $V \leftarrow I[i - 1:i + 2, j - 1:j + 2]$

▷ vecindad 3×3 centrada en (i, j)

4: $I'[i, j] \leftarrow f(V)$

5: **end for**

6: **end for**

7: **return** I'

Para el kernel gaussiano, $f(V) = (\sum V \odot K) / \sum K$, con $K = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$; para Sobel, $f(V) = \sum V \odot K_x$ (análogamente para K_y), y la magnitud de borde reportada es $|G_x| + |G_y|$.

4.2.2. Resultados

Las Figuras 3 y 4 muestran el resultado esperado desde la teoría del operador de Sobel: el kernel horizontal (K_x) aísla los bordes *verticales* de la T (los dos lados del asta y los extremos verticales de la barra superior), el kernel vertical (K_y) aísla los bordes *horizontales* (el borde superior de la barra

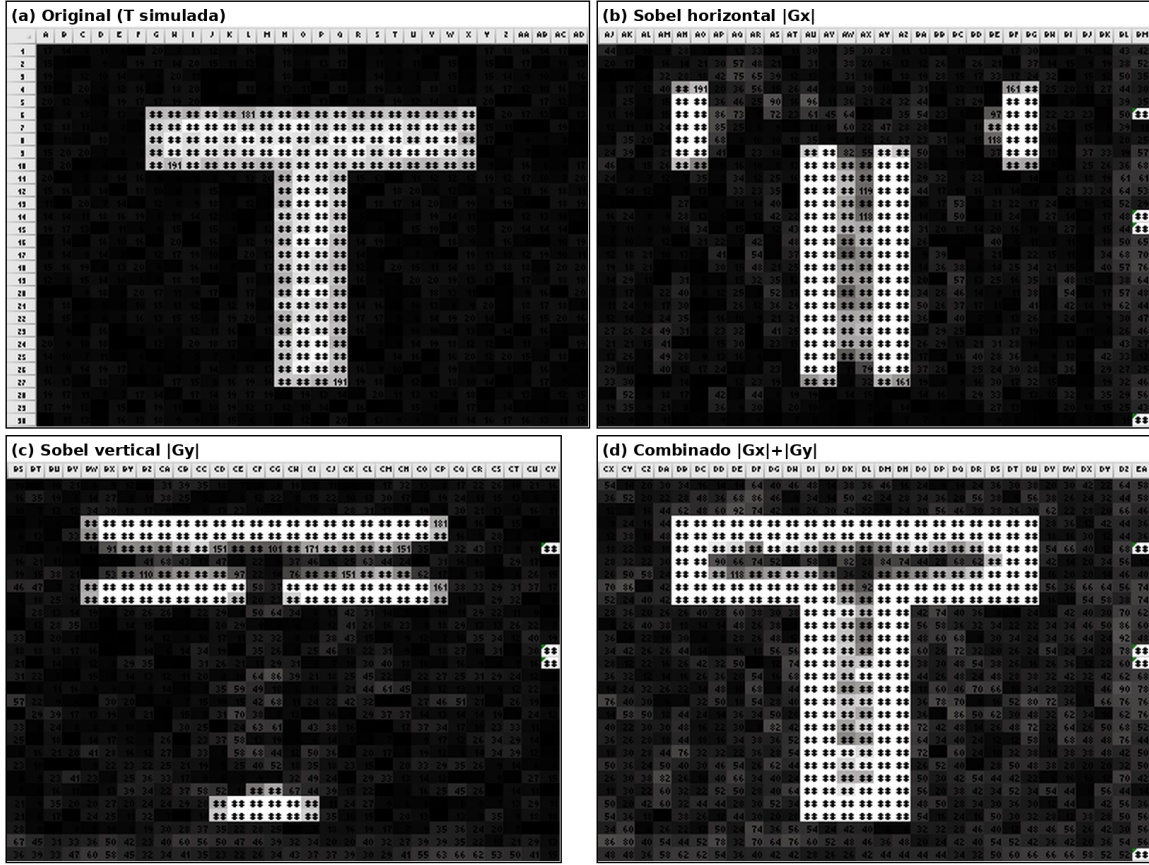


Figura 3: Pipeline sin suavizado previo: (a) matriz original con forma de letra "T" (fondo 0–20, borde de transición 180–220, interior 220–255); (b) respuesta de Sobel horizontal $|G_x|$; (c) respuesta de Sobel vertical $|G_y|$; (d) magnitud combinada $|G_x| + |G_y|$. Capturas de la región válida 28×28 de la hoja de cálculo, con formato condicional en escala de grises.

y el borde inferior a los lados del asta), y la combinación $|G_x| + |G_y|$ traza el contorno completo de la letra, incluyendo tanto sus 4 vértices convexos como sus 2 vértices cóncavos, mientras que el fondo aleatorio de bajo nivel (0–20) queda prácticamente suprimido en comparación.

¿Por qué la imagen filtrada tiene valores por encima de 255? A diferencia de un kernel de suavizado (cuyos pesos suman 1 y por lo tanto producen un promedio ponderado que nunca puede superar el máximo de la vecindad), los kernels de Sobel tienen pesos que suman 0 y están diseñados para responder a *diferencias* de intensidad, no a un promedio. Su salida es una derivada discreta, no un valor de intensidad acotado: con pesos máximos de ± 4 por kernel, la respuesta cruda de cada dirección puede alcanzar en teoría hasta $4 \times 255 = 1020$, y la magnitud combinada $|G_x| + |G_y|$ puede llegar hasta el doble de eso. En la práctica, sobre la región válida 28×28 , el máximo verificado con la función MAX de la hoja de cálculo es de **1238** en la variante sin suavizado (Figura 3d) y de **885** en la variante con suavizado gaussiano previo (Figura 4d); en ambos casos varias celdas del borde de la T superan además la capacidad de columna de la hoja de cálculo (mostrando ### en Excel), lo cual tampoco es un error, sino la misma consecuencia de mostrar un valor más ancho que la columna. Estos valores no son errores: son la magnitud de un gradiente, no un pixel, y para desplegarlos como imagen visualizable habría que normalizarlos o recortarlos (*clipping*) al rango $[0, 255]$; aquí

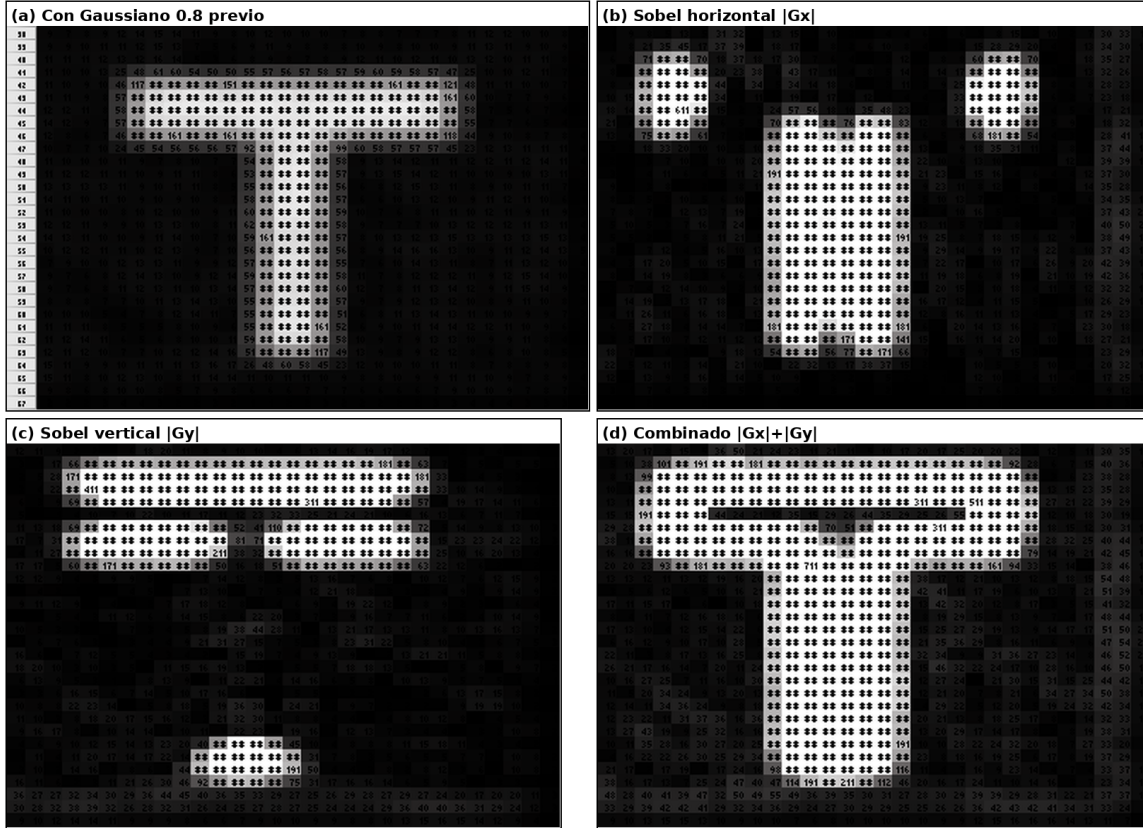


Figura 4: Mismo pipeline, aplicando primero un suavizado gaussiano 3×3 ($\sigma \approx 0,8$) sobre la matriz original: (a) imagen suavizada (nótese que conserva el rango de valores pero introduce decimales y ensancha el borde de transición); (b)–(c) respuesta de Sobel horizontal y vertical sobre la imagen suavizada; (d) magnitud combinada.

se reportan sin recortar porque el valor crudo es precisamente lo que permite comparar de forma directa ambos *pipelines*.

Esa comparación deja además un hallazgo cuantitativo relevante: suavizar antes de derivar (Gaussiano $\rightarrow$ Sobel) atenúa la magnitud máxima de la respuesta de borde en un 28 % (de 1238 a 885 en este ejemplo), el mismo compromiso ruido-vs-nitidez de borde que se cuantifica de forma estadística y formal, con réplicas y prueba de hipótesis, en el Reto 4 (Sección 4.4); este reto se limita a establecer el mecanismo y el método de verificación manual, y deja el análisis estadístico de qué filtro es preferible para esa sección, donde sí hay razón (ruido impulsivo con varias réplicas) para aplicar ANOVA.

Durante la verificación de este reto se detectó y corrigió un error de rango en 14 celdas de las hojas de convolución ('SUMPRODUCT' aplicado sobre una vecindad de 2×3 en vez de 3×3 en filas puntuales), documentado junto con su corrección en el historial de control de versiones del repositorio.

4.2.3. Análisis y conclusiones

Los resultados confirman la teoría: cada kernel de Sobel aísla los bordes de la orientación que le corresponde, la combinación reconstruye el contorno completo de la letra – convexo y cóncavo

por igual $-$, y suavizar antes de derivar atenúa esa respuesta de forma medible (Sección anterior). Aplicar los kernels celda por celda en una hoja de cálculo, en vez de invocar directamente `cv2.Sobel` o `cv2.filter2D`, obliga a hacer explícita la operación que esas funciones ejecutan como caja negra: una suma ponderada sobre una vecindad 3×3 (Algoritmo 1). Esa comprensión de primer principio es también la que sostiene, en la línea de investigación doctoral del autor, cualquier decisión de diseño sobre qué descriptor de bordes/esquinas usar en un *pipeline* de SLAM visual.

4.3. Reto 3: Colores

4.3.1. Método

Se buscó una imagen a color con variedad de colores (la escena sintética descrita en la Sección 3) y se leyó con OpenCV. Su forma (**shape**), impresa directamente en consola como pide la guía, es (1536, 2752, 3) (alto, ancho, 3 canales BGR). Mediante inspección visual se definieron, por sus coordenadas, 6 regiones de interés (ROI) de 80×80 píxeles: 5 homogéneas, una por cada color sólido presente en la escena, y una sexta región deliberadamente heterogénea (mezcla de colores y texturas), tal como pide la guía. El Algoritmo 2 resume el procedimiento de extracción, dibujo del recuadro sobre la imagen original, y caracterización estadística por canal.

Algorithm 2 Extracción y caracterización estadística de regiones de interés

Require: Imagen a color I (canales B, G, R); lista de regiones $R = \{(x_1, y_1, x_2, y_2)\}$

Ensure: Media y desviación estándar por canal para cada región; imagen anotada I_a

```

1:  $I_a \leftarrow \text{copia}(I)$ 
2: for cada región  $r = (x_1, y_1, x_2, y_2) \in R$  do
3:    $P \leftarrow I[y_1:y_2, x_1:x_2]$  ▷ parche de la región
4:   for cada canal  $c \in \{B, G, R\}$  do
5:      $\mu_{r,c} \leftarrow \text{media}(P[:, :, c])$ 
6:      $\sigma_{r,c} \leftarrow \text{desv. estándar}(P[:, :, c])$ 
7:   end for
8:   dibujar_rectángulo( $I_a, r$ )
9: end for
10: return  $\{\mu_{r,c}\}, \{\sigma_{r,c}\}, I_a$ 

```

4.3.2. Resultados

La Tabla 2 resume la media y desviación estándar por canal BGR de las 6 regiones; la Figura 5 muestra las regiones dibujadas sobre la imagen original.

Cuadro 2: Media y desviación estándar por canal (B, G, R) de las 6 regiones de interés.

Región	Media (B, G, R)	Desv. estándar (B, G, R)	Color dominante
R1 – Azul (homog.)	207.4, 95.5, 1.3	1.9, 2.1, 1.2	Azul saturado
R2 – Verde (homog.)	103.5, 191.0, 8.9	2.7, 2.6, 5.1	Verde saturado
R3 – Rojo (homog.)	22.3, 8.0, 204.2	3.0, 3.9, 3.1	Rojo saturado
R4 – Pared (homog.)	245.1, 241.0, 240.4	1.4, 1.4, 1.5	Blanco/gris claro
R5 – Piso (homog.)	204.8, 202.1, 194.4	3.2, 3.2, 3.3	Gris claro
R6 – Panel+caja (heterog.)	150.4, 90.0, 55.3	71.7, 34.5, 61.6	Mezcla

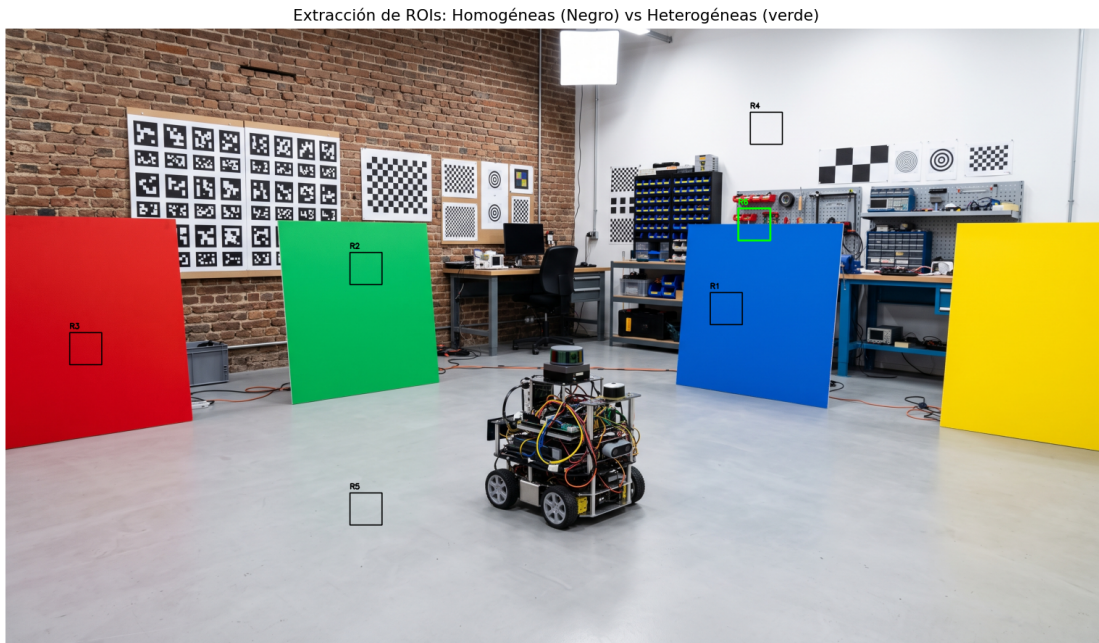


Figura 5: Las 6 regiones de interés dibujadas sobre la imagen original (recuadros negros: regiones homogéneas; recuadro verde: región heterogénea).

La correspondencia entre medias BGR y el color percibido es la esperada: R1 (azul) tiene B alto y R muy bajo; R3 (rojo) tiene R alto y B/G bajos; R2 (verde) tiene G dominante; R4 y R5 (pared y piso) tienen los tres canales altos y muy parecidos entre sí. Las desviaciones estándar de las 5 regiones homogéneas son todas bajas (1,2–5,1), mientras que la región 6 (heterogénea) tiene una desviación estándar de 10 a 30 veces mayor (71,7 en el canal B, frente a un máximo de 3,2 en las homogéneas).

4.3.3. Análisis y conclusiones

La correspondencia entre las medias BGR y el color percibido, y entre la desviación estándar y la homogeneidad visual de cada región, confirma que unos pocos estadísticos por canal bastan para distinguir cuantitativamente regiones de color sólido de una región mixta, sin necesidad de inspección visual una vez calculados. Dado el origen sintético de la imagen (Sección 3), este reto debe entenderse como una prueba de concepto del *pipeline* de extracción y caracterización de regiones (Algoritmo 2); su validación sobre fotografías reales queda como trabajo futuro (Sección 6). El mismo principio – una región con desviación estándar baja es un candidato estable para seguimiento entre *frames*, mientras que una con desviación estándar alta es una región ambigua – es, además, la base de la caracterización de *landmarks* candidatos en SLAM visual, línea de trabajo doctoral del autor.

4.4. Reto 4: Convoluciones para filtrado

4.4.1. Método

Sobre una región de 300×300 píxeles recortada de la misma imagen (Sección 3), partiendo de la imagen limpia se inyectó ruido *salt & pepper* al 4 % de forma controlada. Siguiendo la guía, se buscaron e implementaron sin OpenCV cuatro convoluciones adecuadas para eliminar este tipo de ruido – media, moda, mediana y gaussiana, como filtros de suavizado 3×3 (Algoritmo 1 con $f \in \{\text{media, moda, mediana, gaussiana}\}$, esta última reutilizando el mismo kernel del Reto 2) –, y se compararon contra las funciones propias de OpenCV (`cv2.medianBlur`, recomendada por la guía, y `cv2.GaussianBlur`). Más allá de la comparación cualitativa pedida por la guía, la calidad de cada filtro se evaluó también de forma cuantitativa con el índice de similitud estructural (SSIM) [Wang et al., 2004] respecto a la imagen sin ruido, sobre 10 réplicas independientes de ruido por filtro, seguido de un ANOVA de un factor (el filtro, con 4 niveles) y una prueba post-hoc de Tukey HSD para identificar qué pares de filtros difieren significativamente entre sí.

4.4.2. Resultados

Cuadro 3: SSIM medio y varianza por filtro (10 réplicas).

Filtro	SSIM medio	Varianza
Media	0.6272	0.000009
Moda	0.6475	0.000031
Mediana	0.9255	0.000000
Gaussiana	0.6265	0.000009

El ANOVA de un factor resultó altamente significativo: $F = 15801,59$, $p = 3,40 \times 10^{-56}$. La Tabla 4 presenta las 6 comparaciones post-hoc de Tukey HSD.

Cuadro 4: Comparaciones post-hoc de Tukey HSD entre los 4 filtros ($\alpha = 0,05$).

Comparación	Dif. de medias	p -ajustado	¿Significativa?
Gaussiana vs. Media	0.0007	0.973	No
Gaussiana vs. Mediana	0.2990	$< 0,001$	Sí
Gaussiana vs. Moda	0.0210	$< 0,001$	Sí
Media vs. Mediana	0.2983	$< 0,001$	Sí
Media vs. Moda	0.0203	$< 0,001$	Sí
Mediana vs. Moda	-0,2780	$< 0,001$	Sí

La Figura 6 muestra, de forma cualitativa, el mismo patrón: el ruido desaparece casi por completo con la mediana (manual y OpenCV), mientras que persiste claramente con media y gaussiana.

Adicionalmente, la Tabla 5 compara el costo computacional de la implementación manual frente a OpenCV.

4.4.3. Análisis y conclusiones

El hallazgo central, pedido por la guía como análisis y conclusión de este reto, es que **gaussiana y media son estadísticamente indistinguibles** ($p = 0,973$), y ambas son muy inferiores a la

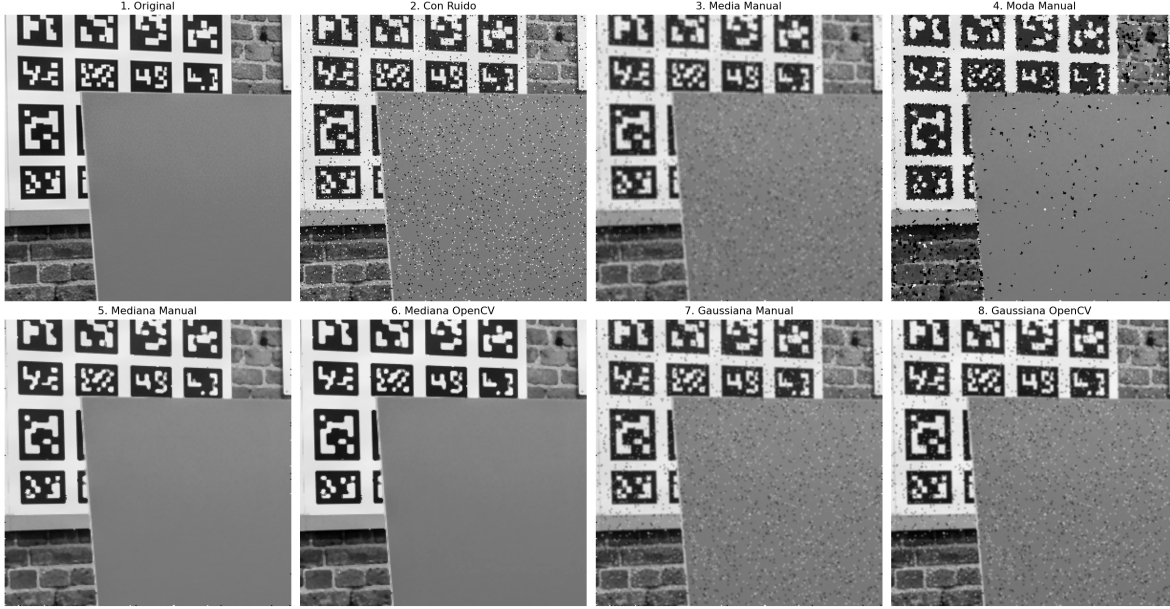


Figura 6: Comparación visual: original, con ruido, y los 4 filtros manuales frente a sus equivalentes de OpenCV (mediana, gaussiana).

Cuadro 5: Tiempo de ejecución: implementación manual vs. OpenCV.

Filtro	Manual (s)	OpenCV (s)	Aceleración
Mediana	0.8433	0.0001	$\sim 5770\times$
Gaussiana	0.1562	0.0007	$\sim 217\times$

mediana. Esto es el resultado teóricamente esperado: media y gaussiana son filtros *lineales* que promedian los valores de la vecindad, incluyendo los valores extremos (0 o 255) que introduce el ruido *salt & pepper*, de modo que el ruido se esparce en vez de eliminarse; la mediana es un filtro *no lineal* (estadístico de orden) que, al tomar el valor central de la vecindad ordenada, descarta directamente esos extremos. Este resultado, bien conocido en la literatura de procesamiento de imágenes, se demuestra aquí empíricamente mediante una prueba de hipótesis formal (ANOVA y Tukey HSD), más allá de lo pedido por la guía, y no solo por inspección visual. La comparación manual-vs-OpenCV, además, confirma que la implementación manual reproduce cualitativamente el mismo resultado que `cv2.medianBlur` y `cv2.GaussianBlur`, a un costo computacional varios órdenes de magnitud mayor.

De los cuatro retos, este es el que más se acerca a un experimento DOE completo: un factor (el filtro, 4 niveles), una variable de respuesta cuantitativa (SSIM) medida con réplicas, un ANOVA para establecer si el factor importa, y una prueba post-hoc para identificar qué niveles difieren entre sí. Ese mismo esqueleto metodológico (factor = algoritmo SLAM, niveles = algoritmos candidatos, respuesta = métrica de error/precisión, réplicas = corridas bajo distintas condiciones) es el que la línea de investigación doctoral del autor busca formalizar más adelante (Sección 6).

5. Discusión

Tomados en conjunto, los cuatro retos cubren la cadena completa de operaciones básicas de visión por computador clásica que pedía la guía: representación matricial de una imagen y caracterización estadística, manual y vectorizada, de sus valores (Reto 1); el operador de convolución y los kernels de borde y de suavizado, aplicados celda a celda para hacerlos explícitos (Reto 2); la representación de color y su caracterización estadística por región (Reto 3); y el filtrado de ruido impulsivo, comparando filtros lineales y no lineales tanto de forma cualitativa como con una prueba de hipótesis formal (Reto 4). En los cuatro casos la implementación manual (sin librerías especializadas) se verificó contra su equivalente vectorizado o de OpenCV, lo que permitió confirmar tanto la corrección de las fórmulas usadas como la magnitud de la ganancia en desempeño que ofrecen esas librerías.

Del conjunto también se desprende una progresión natural de complejidad del análisis solicitado: de una comparación de tiempos (Reto 1), a una verificación de comportamiento esperado por inspección (Reto 2), a una correspondencia cuantitativa entre estadísticos y percepción visual (Reto 3), hasta una prueba de hipótesis formal con réplicas para decidir entre alternativas (Reto 4) – este último, el más cercano en espíritu al tipo de comparación de "tratamientos" que estructura un Diseño de Experimentos.

6. Conclusiones y trabajo futuro

Los cuatro retos de la Guía 1 se resolvieron con implementaciones propias, verificadas de forma independiente contra sus equivalentes vectorizados o de OpenCV, y con el análisis y las conclusiones solicitados por la guía en cada caso. Como trabajo futuro inmediato de esta entrega, se plantea: (1) diseñar y evaluar, en el Reto 2, un kernel de convolución propio adicional a los estándar aquí usados (Sobel, gaussiano), tal como sugiere la guía; y (2) validar las conclusiones de los Retos 3 y 4 sobre una fotografía real además de la escena sintética usada aquí.

Más allá de la guía, estos cuatro retos motivan al autor a formalizar, como parte de su investigación doctoral, un marco metodológico basado en Diseño de Experimentos (DOE) [Montgomery, 2019] para evaluar el desempeño de algoritmos SLAM [Cadena et al., 2016, Mur-Artal and Tardós, 2017]. Varias piezas ya quedaron ensayadas aquí a pequeña escala: medición reproducible de tiempos y caracterización estadística de variables de respuesta (Reto 1), comprensión de primer principio de los operadores de convolución que alimentan cualquier extractor de *features* visual (Reto 2), caracterización estadística de regiones/*landmarks* (Reto 3), y sobre todo el esquema de ANOVA y prueba post-hoc de Tukey del Reto 4, que es en los hechos un prototipo funcional en miniatura del tipo de experimento – factor = algoritmo SLAM, niveles = algoritmos candidatos, respuesta = métrica de error/precisión o tiempo, réplicas = corridas bajo distintas condiciones – que se buscará aplicar a algoritmos SLAM completos en ese trabajo doctoral, quedando la definición formal de esos factores, niveles y variables de respuesta como una línea de trabajo separada de esta entrega.

Referencias

- Milton Alvarado. slam-doe-vision-retos: Estudios piloto de vision por computador para un marco doe de evaluacion de algoritmos slam, 2026. URL <URL_DEL_REPOSITORIO>. Repositorio de codigo y datos. Version en preparacion; actualizar URL y version/DOI antes del envio final.
- Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, Jose Neira, Ian Reid, and John J. Leonard. Past, present, and future of simultaneous localization and mapping: Toward

- the robust-perception age. *IEEE Transactions on Robotics*, 32(6):1309–1332, 2016. doi: 10.1109/TRO.2016.2624754.
- Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, New York, NY, USA, 4th edition, 2018.
- Douglas C. Montgomery. *Design and Analysis of Experiments*. Wiley, 10th edition, 2019.
- Raul Mur-Artal and Juan D. Tardós. ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras. *IEEE Transactions on Robotics*, 33(5):1255–1262, October 2017.
- Davide Scaramuzza and Friedrich Fraundorfer. Visual odometry [tutorial]. *IEEE Robotics & Automation Magazine*, 18(4):80–92, December 2011.
- Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, April 2004.